

Kubernetes

Formation Avancée — Jour 2

Module 7

Authentication et Identification

Deux types d'identités

	User Account	Service Account
Représente	Un humain ou un système CI/CD	Un processus/pod dans le cluster
Créé par	Externe (certificats, OIDC provider)	Kubernetes (API)
Scope	Global (cluster-wide)	Namespaced
Stocké dans	etcd (non en tant que ressource K8s)	etcd (ressource K8s)
Token	Certificat, OIDC token, webhook	JWT monté automatiquement dans le Pod

```
# Service Account – géré par Kubernetes
kubectl create serviceaccount mon-sa -n dev
kubectl get serviceaccounts -n dev

# Un SA "default" existe dans chaque namespace
kubectl get sa default -n dev -o yaml
```

Authentification par certificats X.509

Workflow complet :

1. Créer la clé privée
`openssl genrsa -out alice.key 2048`
2. Créer la demande de certificat (CSR)
`openssl req -new -key alice.key -out alice.csr \`
`-subj "/CN=alice/O=dev-team"`

└─ Certificate Signing Request

└─ Nom de l'utilisateur (Common Name)

└─ Groupe(s) de l'utilisateur
3. Soumettre le CSR à Kubernetes
(via l'API CertificateSigningRequest)
4. Un admin approuve le CSR
`kubectl certificate approve alice-csr`
5. Le certificat signé est récupéré
6. `kubectl` est configuré avec le certificat

Certificats X.509 – Commandes complètes

```
# 1. Clé privée
openssl genrsa -out alice.key 2048

# 2. CSR
openssl req -new -key alice.key -out alice.csr \
  -subj "/CN=alice/O=dev-team/O=viewers"

# 3. Soumettre à K8s
cat <<EOF | kubectl apply -f -
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: alice-csr
spec:
  request: $(cat alice.csr | base64 | tr -d '\n')
  signerName: kubernetes.io/kube-apiserver-client
  expirationSeconds: 86400      # 24h
  usages:
    - client auth
EOF

# 4. Approuver
kubectl certificate approve alice-csr

# 5. Récupérer le certificat signé
kubectl get csr alice-csr -o jsonpath='{.status.certificate}' | base64 -d > alice.crt

# 6. Configurer kubectl
kubectl config set-credentials alice \
  --client-certificate=alice.crt --client-key=alice.key
kubectl config set-context alice-ctx \
  --cluster=kind-formation-k8s --user=alice --namespace=dev
kubectl config use-context alice-ctx
```

Tokens de Service Account

```
# K8s 1.24+ : tokens éphémères (recommandé)
kubectl create token mon-sa -n dev --duration=24h

# Utiliser le token
TOKEN=$(kubectl create token mon-sa -n dev)
kubectl get pods -n dev \
  --token="$TOKEN" \
  --server=https://$(kubectl config view -o jsonpath='{.clusters[0].cluster.server}')
```

Token monté automatiquement dans les Pods :

```
# Chemin par défaut : /var/run/secrets/kubernetes.io/serviceaccount/
# Contient : token, ca.crt, namespace

spec:
  serviceAccountName: mon-sa
  automountServiceAccountToken: true    # false pour désactiver

# Depuis l'intérieur d'un pod
cat /var/run/secrets/kubernetes.io/serviceaccount/token
```

Intégration OIDC (Keycloak, Azure AD, Google)

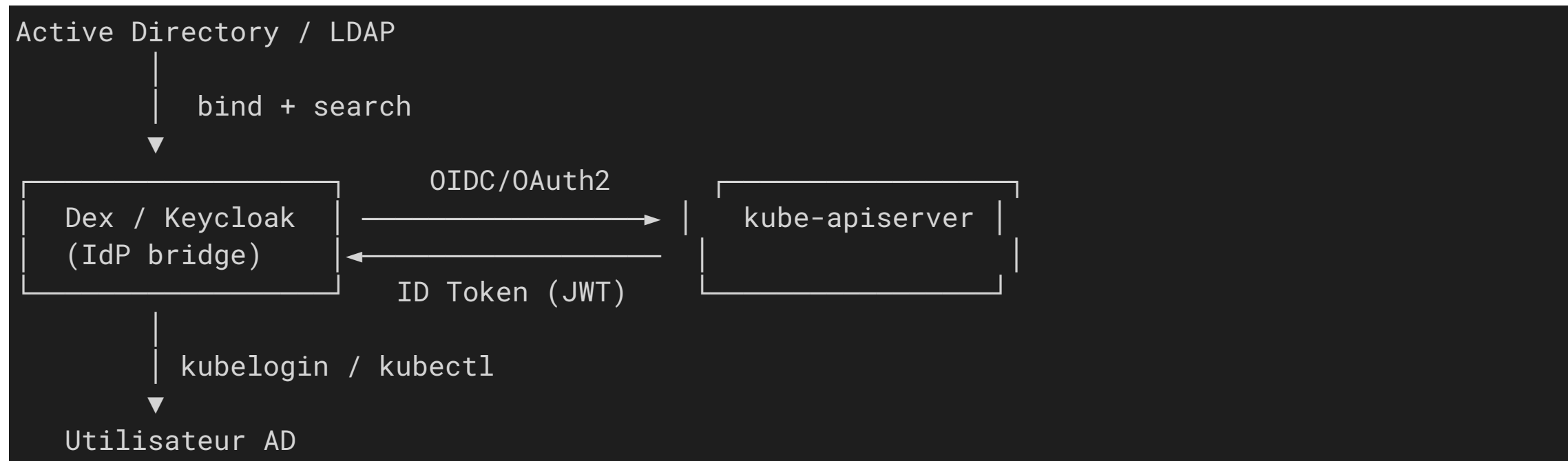
```
# Configuration de l'API Server
apiVersion: kubeadm.k8s.io/v1beta3
kind: ClusterConfiguration
apiServer:
  extraArgs:
    oidc-issuer-url: "https://keycloak.corp.com/realms/kubernetes"
    oidc-client-id: "kubernetes-client"
    oidc-username-claim: "preferred_username"
    oidc-groups-claim: "groups"
    oidc-username-prefix: "oidc:"
    oidc-groups-prefix: "oidc:"
```

Flux OIDC :

1. Utilisateur s'authentifie auprès de l'IdP (Keycloak, Azure AD)
2. IdP retourne un ID Token (JWT)
3. kubectl envoie le token avec chaque requête
4. API Server valide le JWT auprès de l'IdP
5. Extrait username et groups du token
6. RBAC autorise ou refuse

Intégration LDAP / Active Directory

Kubernetes **ne supporte pas LDAP nativement**. L'intégration se fait via un **IdP intermédiaire** qui parle LDAP et expose un endpoint OIDC.



Intégration LDAP / Active Directory

Exemple avec Dex (connector LDAP) :

```
# dex-config.yaml
connectors:
- type: ldap
  id: ldap
  name: Corporate LDAP
  config:
    host: ldap.corp.com:636
    insecureNoSSL: false
    bindDN: cn=dex,ou=service-accounts,dc=corp,dc=com
    bindPW: "$DEX_LDAP_BIND_PW"
    userSearch:
      baseDN: ou=users,dc=corp,dc=com
      filter: "(objectClass=person)"
      username: sAMAccountName
      idAttr: DN
      emailAttr: mail
      nameAttr: displayName
    groupSearch:
      baseDN: ou=groups,dc=corp,dc=com
      filter: "(objectClass=group)"
      userMatchers:
```

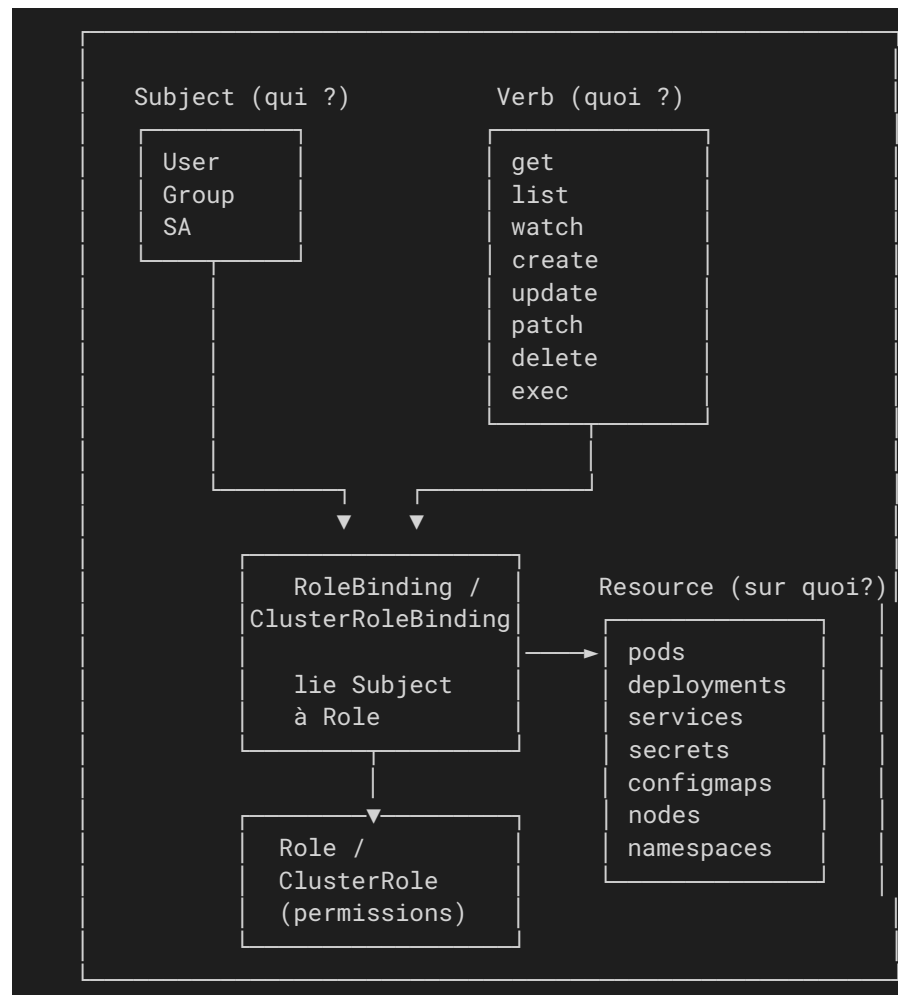
Résumé des méthodes d'authentification

Méthode	Cas d'usage	Sécurité	Complexité
Certificats X.509	CI/CD, automation, admin	Forte	Moyenne
Tokens SA	Pods, communication inter-services	Bonne	Faible
OIDC	SSO entreprise (humains)	Très forte	Élevée
Webhook Token	Intégration custom	Variable	Élevée
Static Token/Password	DÉPRÉCIÉ – ne jamais utiliser	Faible	Faible
Bootstrap Tokens	Ajout de nœuds au cluster	Temporaire	Faible

Module 8

Autorisation — RBAC

Le modèle RBAC



Role vs ClusterRole

Role : permissions dans un **seul namespace**

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-manager
  namespace: dev                # Scope : namespace dev uniquement
rules:
- apiGroups: [""]              # Core API group
  resources: ["pods", "pods/log", "pods/exec"]
  verbs: ["get", "list", "watch", "create", "delete"]
- apiGroups: ["apps"]          # apps API group
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "update", "patch"]
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list"]
```

Role vs ClusterRole

ClusterRole : permissions **cluster-wide** (ou réutilisable dans plusieurs namespaces)

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: node-viewer                # Pas de namespace : cluster-wide
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["get", "list", "watch"]
- apiGroups: [""]
  resources: ["namespaces"]
  verbs: ["get", "list"]
```

RoleBinding et ClusterRoleBinding

RoleBinding : lie un Subject à un Role dans un namespace

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: alice-pod-manager
  namespace: dev
subjects:
- kind: User
  name: alice
  apiGroup: rbac.authorization.k8s.io
- kind: Group
  name: dev-team
  apiGroup: rbac.authorization.k8s.io
- kind: ServiceAccount
  name: ci-deployer
  namespace: ci-system
roleRef:
  kind: Role # Ou ClusterRole
```

RoleBinding et ClusterRoleBinding

Astuce : un RoleBinding peut référencer un ClusterRole. Le ClusterRole est alors restreint au namespace du RoleBinding. Cela permet de réutiliser un ClusterRole dans plusieurs namespaces.

Rôles prédéfinis

ClusterRole	Droits	Usage
cluster-admin	Tout sur tout	Super-admin
admin	Tout dans un namespace (sauf quotas/limitranges)	Chef de projet
edit	CRUD sur la plupart des ressources (pas RBAC)	Développeur
view	Lecture seule	Observateur, QA

Rôles prédéfinis

```
# Voir les règles d'un ClusterRole
kubectl describe clusterrole admin

# Assigner rapidement
kubectl create rolebinding alice-admin \
  --clusterrole=admin --user=alice -n dev

# Vérifier les permissions
kubectl auth can-i create pods --as=alice -n dev # yes
kubectl auth can-i delete secrets --as=alice -n production # no
kubectl auth can-i '*' '*' --as=system:serviceaccount:dev:ci-deployer # no

# Lister toutes les permissions d'un utilisateur
kubectl auth can-i --list --as=alice -n dev
```

RBAC — Bonnes pratiques

- **Principe du moindre privilège** : ne donner que les permissions strictement nécessaires
- **Pas de cluster-admin** sauf pour les ops seniors
- **Préférer les Roles** aux ClusterRoles quand possible
- **Utiliser les Groups** plutôt que les Users individuels
- **Auditer régulièrement** :

```
kubectl get rolebindings,clusterrolebindings -A
```
- **Séparer les namespaces** par équipe/environnement
- **Limiter l'accès aux Secrets** : créer un Role dédié

RBAC – Bonnes pratiques

```
# Exemple : Role qui interdit l'accès aux secrets
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: dev-no-secrets
  namespace: dev
rules:
  - apiGroups: ["", "apps", "batch"]
    resources: ["pods", "deployments", "services", "jobs", "configmaps"]
    verbs: ["*"]
  # Pas de règle pour "secrets" → accès refusé par défaut
```

Module 9

Sécurité Réseau et Runtime

Network Policies — Le pare-feu Kubernetes

Par défaut, **tous les Pods peuvent communiquer avec tous** les autres Pods du cluster.

Les **NetworkPolicies** permettent de contrôler le trafic :

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: deny-all-ingress
  namespace: production
spec:
  podSelector: {}                # Tous les pods du namespace
  policyTypes:
    - Ingress                    # Bloquer tout le trafic entrant
    - Egress                     # Bloquer tout le trafic sortant
```

“ **Important** : les NetworkPolicies sont **additives**. Si aucune policy ne sélectionne un pod, tout le trafic est autorisé. Dès qu'une policy sélectionne un pod, seul le trafic explicitement autorisé passe. ”

NetworkPolicy – Règles détaillées

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: api-policy
  namespace: production
spec:
  podSelector:
    matchLabels:
      app: api                # S'applique aux pods app=api
  policyTypes:
    - Ingress
    - Egress
  ingress:
    - from:
        - podSelector:        # Depuis les pods frontend
            matchLabels:
              app: frontend
        - namespaceSelector:  # Depuis le namespace monitoring
            matchLabels:
              name: monitoring
      ports:
        - port: 8080
          protocol: TCP
  egress:
    - to:
        - podSelector:
            matchLabels:
              app: database
      ports:
        - port: 5432
    - to: []                  # Autoriser DNS
      ports:
        - port: 53
```

NetworkPolicy – Patterns courants

1. Default deny all (baseline sécurisée) :

```
podSelector: {}
policyTypes: [Ingress, Egress]
# Aucune règle ingress/egress = tout bloqué
```

2. Autoriser DNS pour tous :

```
podSelector: {}
egress:
  - to: []
    ports: [{port: 53, protocol: UDP}, {port: 53, protocol: TCP}]
policyTypes: [Egress]
```

3. Architecture 3-tiers :

```
Internet → frontend (port 80) → api (port 8080) → database (port 5432)
      Ingress only           Ingress from           Ingress from
      from Ingress           frontend only         api only
      Controller
```


Security Context – Contraintes d'exécution

```
apiVersion: v1
kind: Pod
metadata:
  name: secure-pod
spec:
  securityContext:
    # Niveau Pod
    runAsNonRoot: true      # Interdit l'exécution en root
    runAsUser: 1000         # UID forcé
    runAsGroup: 3000        # GID forcé
    fsGroup: 2000           # GID pour les volumes montés
    seccompProfile:
      type: RuntimeDefault  # Profil seccomp
    supplementalGroups: [4000] # Groupes supplémentaires
  containers:
    - name: app
      image: mon-app:latest
      securityContext:
        # Niveau Conteneur
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true  # Filesystem en lecture seule
        capabilities:
          drop: ["ALL"]                # Supprimer toutes les capabilities
          add: ["NET_BIND_SERVICE"]    # Ajouter seulement ce qui est nécessaire
        privileged: false              # Jamais en prod !
      volumeMounts:
        - name: tmp
          mountPath: /tmp              # Écriture possible ici
```

Pod Security Standards (PSS)

Kubernetes 1.25+ intègre des **Pod Security Standards** au niveau du namespace :

Level	Description	Restrictions principales
Privileged	Aucune restriction	Tout autorisé
Baseline	Restrictions minimales	Pas de hostNetwork, pas de privileged, pas de hostPID
Restricted	Bonnes pratiques strictes	Non-root, readOnlyRootFS, drop ALL capabilities, seccomp

```
# Appliquer au namespace
kubectl label namespace production \
  pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted \
  pod-security.kubernetes.io/audit=restricted

# Modes : enforce (bloque), warn (avertit), audit (log)
```

“ Remplace les anciennes PodSecurityPolicies (PSP), supprimées en K8s 1.25. ”

Sécurité des images

La chaîne de sécurité des images couvre **3 niveaux** : construction, stockage et déploiement.

1. Scan de vulnérabilités avec Trivy :

```
# Scan d'une image locale
trivy image mon-app:latest

# Scan avec seuil de sévérité (fail si HIGH ou CRITICAL)
trivy image --exit-code 1 --severity HIGH,CRITICAL mon-app:latest

# Scan dans la CI (GitLab CI / GitHub Actions)
trivy image --format json --output trivy-report.json mon-app:$TAG
```

Sécurité des images

```
mon-app:latest (debian 11.6)
```

```
Total: 42 (UNKNOWN: 0, LOW: 20, MEDIUM: 18, HIGH: 3, CRITICAL: 1)
```

Library	Vulnerability	Severity	Fix
openssl	CVE-2023-0286	CRITICAL	3.0.8-1
curl	CVE-2023-23916	HIGH	7.88.1-10

Sécurité des images

2. Bonnes pratiques de construction :

```
# Image de base minimale et versionnée (pas de :latest)
FROM debian:12.5-slim

# Utilisateur non-root
RUN useradd -u 1000 -m appuser
USER appuser

# Pas de secrets dans les layers
# ✗ COPY secrets.env .
# ✔ Utiliser des Secrets K8s ou Vault
```

3. Signature et vérification (Cosign) :

```
# Signer une image après build
cosign sign --key cosign.key registry/mon-app:v1.2.3

# Vérifier la signature avant déploiement
cosign verify --key cosign.pub registry/mon-app:v1.2.3
```

Sécurité des images

4. Politique dans K8s (Admission Webhook) :

- **Kyverno** :

```
require-image-signature
```

```
,
```

```
disallow-latest-tag
```

- **OPA/Gatekeeper** : politiques sur les registries autorisées
- **ImagePolicyWebhook** : valide les images avant création de Pod

“ **Règle d'or** : jamais d'image

```
:latest
```

en production, toujours scanner avant push.

”

Exercice 03 : RBAC + NetworkPolicies →

```
exercices/jour2/exercice-03-rbac-netpol.md
```

Module 10

Stratégies de Déploiement

Vue d'ensemble des stratégies

Stratégie	Downtime	Risque	Rollback	Complexité
Recreate	Oui	Élevé	Manuel	Très faible
Rolling Update	Non	Moyen	oc rollout undo	Faible
Blue/Green	Non	Faible	Basculer le Service	Moyenne
Canary	Non	Très faible	Supprimer le canary	Moyenne
A/B Testing	Non	Très faible	Selon métriques	Élevée

Rolling Update – Détails de configuration

```
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 25%           # Pods en plus (absolue ou %)
      maxUnavailable: 25%    # Pods en moins (absolue ou %)
```

maxSurge	maxUnavailable	Comportement
1	0	Le plus sûr : crée 1 nouveau avant de supprimer 1 ancien
25%	25%	Rapide : 25% de pods en plus, 25% en moins simultanément
0	1	Économe : pas de pod supplémentaire
100%	0	Instantané : tous les nouveaux pods d'abord

Rolling Update — Détails de configuration

```
# Déclencher
kubectl set image deployment/mon-app app=mon-app:v2

# Suivre en temps réel
kubectl rollout status deployment/mon-app

# Voir les ReplicaSets (ancien et nouveau)
kubectl get rs -l app=mon-app

# Rollback instantané
kubectl rollout undo deployment/mon-app
```

Blue/Green — Déploiement parallèle

```
# Blue (version actuelle, en production)
kubectl apply -f deployment-blue.yaml # app=mon-app, version=blue

# Service pointe vers Blue
kubectl apply -f - <<EOF
apiVersion: v1
kind: Service
metadata:
  name: mon-app
spec:
  selector:
    app: mon-app
    version: blue # ← pointe vers Blue
  ports:
    - port: 80
      targetPort: 8080
EOF

# Green (nouvelle version, déployée à côté)
kubectl apply -f deployment-green.yaml # app=mon-app, version=green

# Test du Green (via port-forward ou Service temporaire)
kubectl port-forward deployment/mon-app-green 8081:8080
curl localhost:8081/healthz

# Bascule instantanée
```

Canary — Déploiement progressif

```
# Stable (90% du trafic)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app-stable
spec:
  replicas: 9                # 90% du trafic
  selector:
    matchLabels:
      app: mon-app
  template:
    metadata:
      labels:
        app: mon-app        # Même label que le canary
    spec:
      containers:
        - name: app
          image: mon-app:v1

# Canary (10% du trafic)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mon-app-canary
spec:
  replicas: 1                # 10% du trafic
  selector:
    matchLabels:
      app: mon-app          # MÊME label → même Service
  template:
    metadata:
      labels:
```

HPA – Horizontal Pod Autoscaler

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: mon-app-hpa
  namespace: production
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: mon-app
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 100
          periodSeconds: 60
    scaleDown:
```

VPA – Vertical Pod Autoscaler

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: mon-app-vpa
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: mon-app
  updatePolicy:
    updateMode: "Auto"           # Off | Initial | Recreate | Auto
  resourcePolicy:
    containerPolicies:
      - containerName: app
        minAllowed:
          cpu: "50m"
          memory: "64Mi"
        maxAllowed:
```

Rollback — Revenir en arrière avec kubectl

Kubernetes conserve l'**historique des révisions** d'un Deployment (par défaut : 10).

```
# Voir l'historique
kubectl rollout history deployment/mon-app

# REVISION  CHANGE-CAUSE
# 1          kubectl apply -f deployment.yaml
# 2          kubectl set image deployment/mon-app app=mon-app:v2
# 3          kubectl set image deployment/mon-app app=mon-app:v3 ← current

# Détail d'une révision
kubectl rollout history deployment/mon-app --revision=2

# Rollback vers la révision précédente
kubectl rollout undo deployment/mon-app

# Rollback vers une révision spécifique
kubectl rollout undo deployment/mon-app --to-revision=1

# Suivre le rollback
```

Module 11

ArgoCD — GitOps

Principes du GitOps

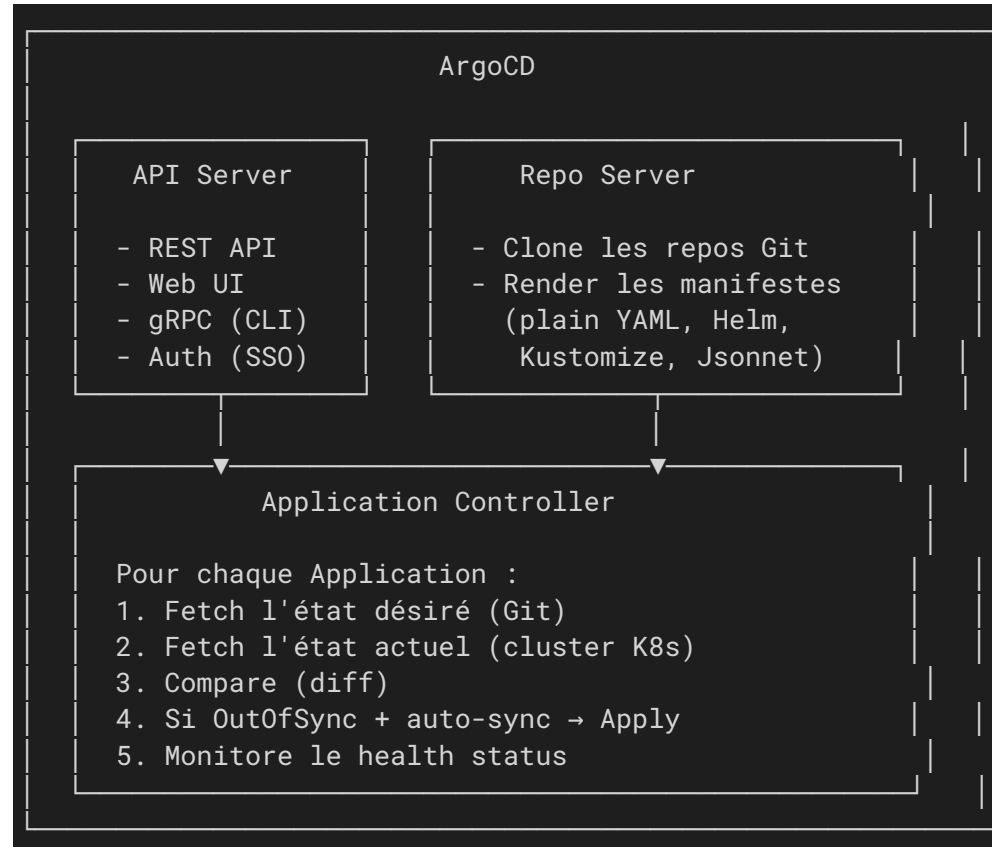
GitOps est une méthodologie opérationnelle :

Principe	Description
Déclaratif	L'infrastructure et les apps sont décrites en YAML/JSON dans Git
Versionné	Git = historique complet, audit trail, diff
Automatique	Les changements dans Git sont automatiquement appliqués
Réconciliation	L'agent compare en continu l'état réel vs l'état désiré

Avantages :

- Rollback =
`git revert`
- Code review sur l'infrastructure (Pull Request)
- Reproductibilité totale (rebuild from scratch)
- Séparation des responsabilités (dev push le code, ArgoCD déploie)

Architecture ArgoCD



Installation d'ArgoCD sur Kind

```
# Installer
kubectl create namespace argocd
kubectl apply -n argocd \
  -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml

# Attendre le démarrage
kubectl wait --for=condition=available deployment/argocd-server \
  -n argocd --timeout=180s

# Récupérer le mot de passe admin
kubectl -n argocd get secret argocd-initial-admin-secret \
  -o jsonpath="{.data.password}" | base64 -d

# Accéder à l'UI
kubectl port-forward svc/argocd-server -n argocd 8080:443 &
# → https://localhost:8080

# Login CLI
argocd login localhost:8080 --insecure --username admin --password <pwd>
```

Application ArgoCD – Manifest détaillé

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mon-app-production
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io    # Cleanup à la suppression
spec:
  project: default
  source:
    repoURL: https://github.com/mon-org/k8s-manifests.git
    targetRevision: main                        # Branche, tag, ou commit SHA
    path: environments/production               # Chemin dans le repo
    # Pour Helm :
    # helm:
    #   valueFiles:
    #     - values-production.yaml
    #   parameters:
    #     - name: image.tag
    #       value: "v2.3.1"
  destination:
    server: https://kubernetes.default.svc      # Cluster cible
    namespace: production
  syncPolicy:
    automated:
      prune: true                             # Supprimer les ressources orphelines
      selfHeal: true                           # Re-synchroniser si drift détecté
      allowEmpty: false                        # Ne pas supprimer tout si le repo est vide
    syncOptions:
      - CreateNamespace=true
      - PrunePropagationPolicy=foreground
      - PruneLast=true
  retry:
    limit: 5
```

ArgoCD – Sync et Health Status

Sync Status :

Status	Signification	Action
Synced	Cluster == Git	Rien à faire
OutOfSync	Cluster != Git	Sync nécessaire
Unknown	Impossible de déterminer	Vérifier la connectivité

Health Status :

Status	Signification
Healthy	Toutes les ressources OK
Progressing	Déploiement en cours (rollout)
Degraded	Erreurs détectées (pods en CrashLoop, etc.)
Suspended	Ressource intentionnellement pausée
Missing	Ressource dans Git mais absente du cluster

ArgoCD – Sync et Health Status

```
argocd app get mon-app-production  
argocd app diff mon-app-production  
argocd app sync mon-app-production  
argocd app history mon-app-production  
argocd app rollback mon-app-production <revision>
```

Gestion multi-environnements

```
k8s-manifests/
├── base/                                # Manifestes communs
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
├── environments/
│   ├── dev/                            # patches pour dev
│   │   ├── kustomization.yaml
│   │   └── replicas-patch.yaml
│   ├── staging/
│   │   ├── kustomization.yaml
│   │   └── replicas-patch.yaml
│   └── production/
│       ├── kustomization.yaml
│       └── replicas-patch.yaml
```

Gestion multi-environnements

Ou avec Helm :

```
helm-charts/  
├── mon-app/  
│   ├── Chart.yaml  
│   ├── values.yaml           # Valeurs par défaut  
│   ├── values-dev.yaml  
│   ├── values-staging.yaml  
│   └── values-production.yaml
```


Module 12

Helm

Helm – Concepts fondamentaux

```
CHART
(package = templates + values)

Chart.yaml    (métadonnées)
values.yaml   (valeurs défaut)
templates/    (manifestes K8s)
charts/       (dépendances)

+
values-prod.yaml (override custom)
=
RELEASE
(instance déployée dans le cluster)
```

Helm — Concepts fondamentaux

- **Chart** : un package Helm (template de manifestes)
- **Release** : une instance déployée d'un chart
- **Repository** : un dépôt de charts (comme npm/Maven)

Commandes Helm essentielles

```
# Gestion des repos
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm search repo bitnami/nginx

# Installation
helm install mon-nginx bitnami/nginx -n dev
helm install mon-nginx bitnami/nginx -f values-custom.yaml -n dev
helm install mon-nginx bitnami/nginx --set replicaCount=3 -n dev

# Mise à jour
helm upgrade mon-nginx bitnami/nginx --set replicaCount=5 -n dev
helm upgrade mon-nginx bitnami/nginx -f values-v2.yaml -n dev

# Rollback
helm rollback mon-nginx 1 -n dev    # Revenir à la révision 1

# Gestion
helm list -n dev                    # Lister les releases
helm status mon-nginx -n dev        # Status d'une release
helm history mon-nginx -n dev        # Historique des révisions
helm get values mon-nginx -n dev     # Valeurs actuelles
helm get manifest mon-nginx -n dev  # Manifestes générés
```

Créer un Chart Helm

```
helm create mon-webapp
```

```
mon-webapp/  
├── Chart.yaml           # Nom, version, description, type  
├── values.yaml          # Valeurs par défaut  
├── charts/              # Sous-charts (dépendances)  
├── templates/  
│   ├── _helpers.tpl     # Fonctions Go template réutilisables  
│   ├── deployment.yaml  # Template de Deployment  
│   ├── service.yaml     # Template de Service  
│   ├── ingress.yaml     # Template d'Ingress  
│   ├── hpa.yaml         # Template HPA  
│   ├── serviceaccount.yaml # Template ServiceAccount  
│   ├── NOTES.txt        # Message post-install  
│   └── tests/  
│       └── test-connection.yaml  
└── .helmignore          # Fichiers à ignorer
```

Templates Helm – Syntaxe Go

```
# templates/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ include "mon-webapp.fullname" . }}
  labels:
    {{- include "mon-webapp.labels" . | nindent 4 }}
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      {{- include "mon-webapp.selectorLabels" . | nindent 6 }}
  template:
    metadata:
      labels:
        {{- include "mon-webapp.selectorLabels" . | nindent 8 }}
    spec:
      containers:
        - name: {{ .Chart.Name }}
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag | default .Chart.AppVersion }}"
          imagePullPolicy: {{ .Values.image.pullPolicy }}
          ports:
            - name: http
              containerPort: {{ .Values.service.port }}
          {{- if .Values.resources }}
          resources:
            {{- toYaml .Values.resources | nindent 12 }}
          {{- end }}
          {{- if .Values.livenessProbe.enabled }}
          livenessProbe:
            httpGet:
```

values.yaml – Configuration par défaut

```
replicaCount: 2

image:
  repository: nginx
  tag: "1.25"
  pullPolicy: IfNotPresent

service:
  type: ClusterIP
  port: 80

ingress:
  enabled: false
  className: nginx
  hosts:
    - host: app.local
      paths:
        - path: /
          pathType: Prefix

resources:
  requests:
    cpu: 100m
    memory: 128Mi
  limits:
    cpu: 500m
    memory: 256Mi

autoscaling:
  enabled: false
  minReplicas: 2
  maxReplicas: 10
  targetCPUUtilization: 70

livenessProbe:
  enabled: true
```

Helm — Fonctions et pipelines

```
# Fonctions courantes
{{ .Values.name | upper }}      # MAJUSCULES
{{ .Values.name | lower }}     # minuscules
{{ .Values.name | quote }}     # "entre guillemets"
{{ .Values.name | default "nginx" }} # Valeur par défaut
{{ .Values.name | required "name requis" }} # Erreur si vide
{{ .Values.data | toYaml | nindent 8 }} # YAML indenté

# Conditions
{{- if .Values.ingress.enabled }}
  # Créer l'Ingress
{{- end }}

# Boucles
{{- range .Values.ingress.hosts }}
  - host: {{ .host | quote }}
    http:
      paths:
        {{- range .paths }}
        - path: {{ .path }}
          pathType: {{ .pathType }}
        {{- end }}
      {{- end }}
```


Helm + ArgoCD – Intégration

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: mon-app-helm
  namespace: argocd
spec:
  source:
    repoURL: https://github.com/mon-org/helm-charts.git
    targetRevision: main
    path: charts/mon-webapp
    helm:
      valueFiles:
        - values.yaml
        - values-production.yaml
      parameters:
        - name: image.tag
          value: "v2.3.1"
        - name: replicaCount
          value: "5"
  destination:
    server: https://kubernetes.default.svc
    namespace: production
  syncPolicy:
    automated:
```

Bonus — Kustomize : alternative à Helm

```
k8s-manifests/  
├── base/  
│   ├── kustomization.yaml  
│   ├── deployment.yaml  
│   ├── service.yaml  
│   └── configmap.yaml  
└── overlays/  
    ├── dev/  
    │   ├── kustomization.yaml  
    │   └── patch-replicas.yaml  
    └── production/  
        ├── kustomization.yaml  
        └── patch-replicas.yaml
```

Bonus — Kustomize : alternative à Helm

```
# base/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml

# overlays/production/kustomization.yaml
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
patchesStrategicMerge:
  - patch-replicas.yaml
namespace: production

# overlays/production/patch-replicas.yaml
apiVersion: apps/v1
kind: Deployment
metadata: {name: mon-app}
```